UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Facultad de Ingeniería – Núcleo MIDIA

Estructuras de Datos y Algoritmos 1

TALLER ESTUDIO – CORTE 3 (ALGORITMOS RECURSIVOS)

Integrantes: Sergio Gómez Quiñones (código 2226785) y Brandon Bonilla
Grupo: 51
Fecha: 04/11/2026

En todo el taller n es el tamaño del segmento que se procesa (n = j − i + 1) y N es el tamaño del arreglo completo (A.length). En la llamada inicial se envía todo el arreglo, así que al final N = n. Las c_1, c_2 y c son constantes (comparaciones, asignaciones, operaciones aritméticas y el return de cada llamada).

PRIMERA PARTE: tiempo de ejecución y complejidad

a) recursivo2 (acota es O(lg n))

recursivo2 avanza i de uno en uno hasta pasar a j. En cada llamada multiplica el mayor entre A[i] y A[j] por lo que retorna la llamada siguiente, y cuando i > j retorna acota(A). Cada llamada que no es caso base hace una comparación, un Math.max, una multiplicación y una sola llamada recursiva con un elemento menos.

$T(n) = O(\lg n)$ si $n = 0$ (caso base: se llama a acota una sola vez)

$T(n) = O(1) + T(n - 1)$ si $n > 0$

Expandiendo: $T(n) = T(n - 1) + c_2 = T(n - 2) + 2c_2 = \ldots = T(n - k) + k \cdot c_2$. El caso base se alcanza cuando k = n:

$T(n) = n \cdot c_2 + c_1 + \lg n$

Como n crece más rápido que lg n, **recursivo2 es O(n)**.

b) recursivo3 (obtenerMayor es O(n))

recursivo3 divide el segmento en tres partes de tamaño n/3 y hace tres llamadas recursivas. Antes de dividir llama a obtenerMayor, que es O(n), y después suma los tres resultados. Es el mismo esquema que OPERA_ALGO de la clase, pero dividiendo en tres en vez de en dos.

$T(n) = O(1)$ si $n = 0$

$T(n) = O(n) + 3T(n/3)$ si $n > 0$

Por árbol: la raíz cuesta cn. En el segundo nivel hay 3 llamadas de tamaño n/3, que cuestan $3 \cdot c(n/3) = cn$. En el tercer nivel hay 9 llamadas de tamaño n/9, que cuestan $9 \cdot c(n/9) = cn$, y así sucesivamente. Cada nivel cuesta cn y hay $\log_3 n$ niveles hasta llegar a segmentos de tamaño 1. Los casos base son del orden de n y cada uno es O(1).

$T(n) = cn \cdot \log_3 n + cn$

Como $\log_3 n = \lg n / \lg 3$ solo cambia por una constante, **recursivo3 es O(n lg n)**.

Si se tiene en cuenta que en el código obtenerMayor recibe el arreglo completo A y no solo el segmento A[i..j], cada llamada costaría N (el tamaño de todo el arreglo) en lugar de n. Como hay del orden de n llamadas, el resultado sería $O(n^2)$.

Observación: al probar el código tal como está en el enunciado con info = [4, 7, 21, 10, 13, 23], JavaScript da "RangeError: Maximum call stack size exceeded". Cuando el segmento tiene 1 o 2 elementos, parte = Math.trunc((j+1−i)/3) = 0 y la tercera llamada es recursivo3(A, i, j) con el mismo segmento, así que nunca llega al caso base. El análisis de arriba supone que la división sí llega al caso base, por ejemplo agregando un caso base para cuando i == j.

c) recursivo4

recursivo4 parte el segmento en dos mitades, hace dos llamadas recursivas y suma lo que retornan. Fuera de las llamadas solo hace operaciones constantes (cálculo de parte, la suma y el return), y en el caso base retorna i.

$T(n) = O(1)$ si $n = 0$

$T(n) = O(1) + 2T(n/2)$ si $n > 0$

Expandiendo: $T(n) = 2T(n/2) + c_2 = 4T(n/4) + 3c_2 = 8T(n/8) + 7c_2 = \dots = 2^kT(n/2^k) + (2^k - 1)c_2$. Con $2^k = n$:

$T(n) = n \cdot T(1) + (n - 1)c_2$

Todo es lineal, entonces **recursivo4 es O(n)**.

Observación: en el código, parte se calcula con Math.trunc((i+j)/2), que es la posición del medio y no la mitad del tamaño (j − i + 1)/2. Además, cuando el segmento tiene un solo elemento parte da 0 y se vuelve a llamar con el mismo segmento. Por eso, igual que recursivo3, el código tal cual se desborda en JavaScript. El análisis supone que la división en dos mitades sí llega al caso base.

SEGUNDA PARTE: implementación de algoritmos recursivos

Las dos funciones están en el archivo TallerCorte3_SergioGomez_BrandonBonilla.js, que se ejecuta con *node TallerCorte3_SergioGomez_BrandonBonilla.js* y trae pruebas. Todas las funciones son recursivas y no usan ciclos.

a) proceso1

Qué hace: cuenta cuántas veces aparece x en el arreglo A desde la posición p hasta el final. Si A[p] es igual a x suma 1 y sigue con p + 1; cuando p se pasa del último índice retorna 0. Con info = [3, 5, 2, 10, 5, 20], proceso1(info, 5, 0) retorna 2.

```
6   // a) proceso1: cuenta cuántas veces aparece x en A desde la posición p hasta el final
7   function proceso1(A, x, p) {
8       if (p > A.length - 1) {
9           return 0;
10      } else {
11          if (A[p] == x) {
12              return 1 + proceso1(A, x, p + 1);
13          } else {
14              return proceso1(A, x, p + 1);
15          }
16      }
17  }
```

Figura 1. Implementación de proceso1 en Visual Studio Code.

Sea m = tamaño(A) − p, la cantidad de posiciones que faltan por revisar. Cada llamada hace comparaciones constantes y una sola llamada recursiva con m − 1.

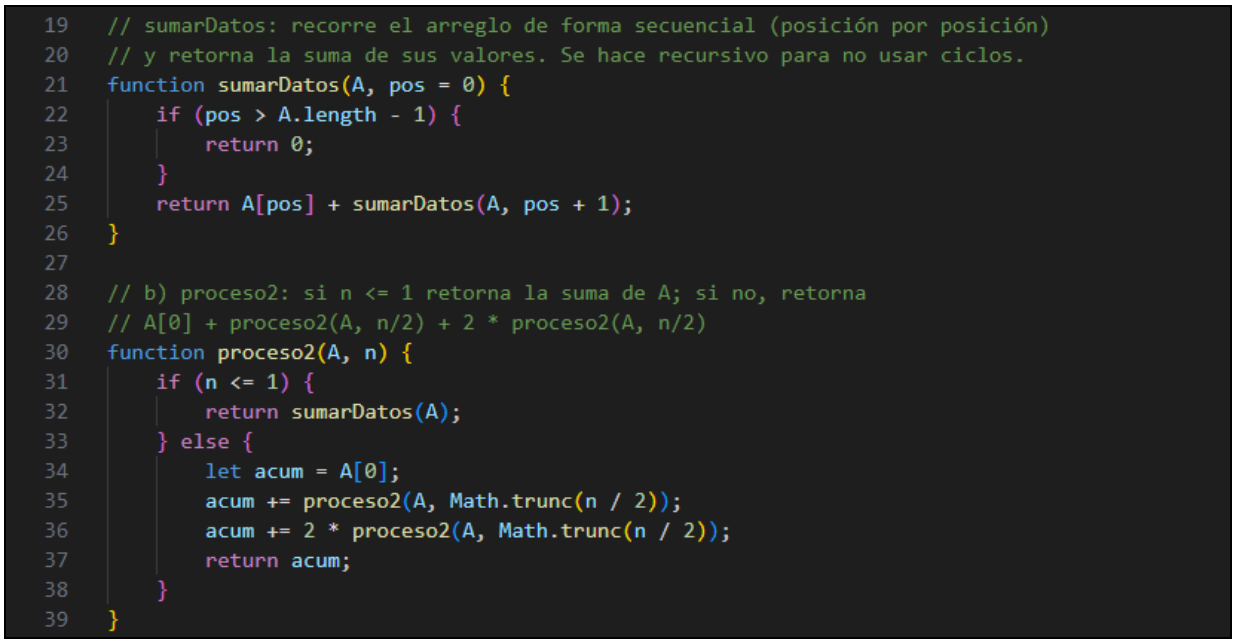
$T(m) = O(1)$ si $m = 0$

$T(m) = O(1) + T(m - 1)$ si $m > 0$

Expandiendo, $T(m) = T(m - k) + k \cdot c_2$ y con $k = m$ queda $T(m) = m \cdot c_2 + c_1$. Como la llamada inicial es con p = 0, m = n y **proceso1 es O(n)**. Al contar las llamadas, con n = 100 hay 101 y con n = 1000 hay 1001.

b) proceso2 (sumarDatos recorre el arreglo de forma secuencial)

Qué hace: si n es menor o igual a 1 retorna la suma de todo el arreglo. Si no, retorna A[0] + proceso2(A, n/2) + 2·proceso2(A, n/2). Las dos llamadas son exactamente iguales, así que en realidad es A[0] + 3·proceso2(A, n/2). Si n se divide k veces hasta llegar a 1, el resultado es $A[0] \cdot (1 + 3 + \ldots + 3^{k-1}) + 3^k \cdot \text{suma}(A)$. Con info = [3, 5, 2, 10, 5, 20] (n = 6 → 3 → 1, k = 2) da 3·4 + 9·45 = 417.



```
// sumarDatos: recorre el arreglo de forma secuencial (posición por posición)
// y retorna la suma de sus valores. Se hace recursivo para no usar ciclos.
function sumarDatos(A, pos = 0) {
    if (pos > A.length - 1) {
        return 0;
    }
    return A[pos] + sumarDatos(A, pos + 1);
}

// b) proceso2: si n <= 1 retorna la suma de A; si no, retorna
// A[0] + proceso2(A, n/2) + 2 * proceso2(A, n/2)
function proceso2(A, n) {
    if (n <= 1) {
        return sumarDatos(A);
    } else {
        let acum = A[0];
        acum += proceso2(A, Math.trunc(n / 2));
        acum += 2 * proceso2(A, Math.trunc(n / 2));
        return acum;
    }
}
```

Figura 2. Implementación de sumarDatos y proceso2 en Visual Studio Code.

sumarDatos se hizo recursivo para no usar ciclos, pero sigue recorriendo el arreglo posición por posición, así que es O(N). En el caso base se suma todo el arreglo A, no una parte, entonces el caso base cuesta c·N.

$T(n) = O(N)$ si $n \leq 1$ (sumarDatos recorre todo el arreglo)

$T(n) = O(1) + 2T(n/2)$ si $n > 1$

Expandiendo: $T(n) = 2^k T(n/2^k) + (2^k - 1)c_2$. Con $2^k = n$:

$T(n) = n \cdot c \cdot N + (n - 1)c_2$

Como N = n, $T(n) \approx c \cdot n^2$, entonces **proceso2 es $O(n^2)$**. Lo comprobamos contando las llamadas (proceso2 más sumarDatos): con n = 256 son 66 303 y con n = 1024 son 1 051 647, que es prácticamente n^2.

Como las dos llamadas recursivas dan el mismo valor, se podría guardar el resultado de una sola llamada en una variable y usarlo dos veces. Así la recurrencia sería $T(n) = T(n/2) + c_2$, la parte recursiva quedaría en O(lg n) y el total en O(n) por la única suma del caso base.

Ejecución

Al ejecutar el archivo con Node.js se imprimen los resultados con info = [3, 5, 2, 10, 5, 20] y se corren las pruebas (casos con el valor repetido, ausente, arreglo vacío, todos iguales y proceso2 con n = 1, 6 y 8).

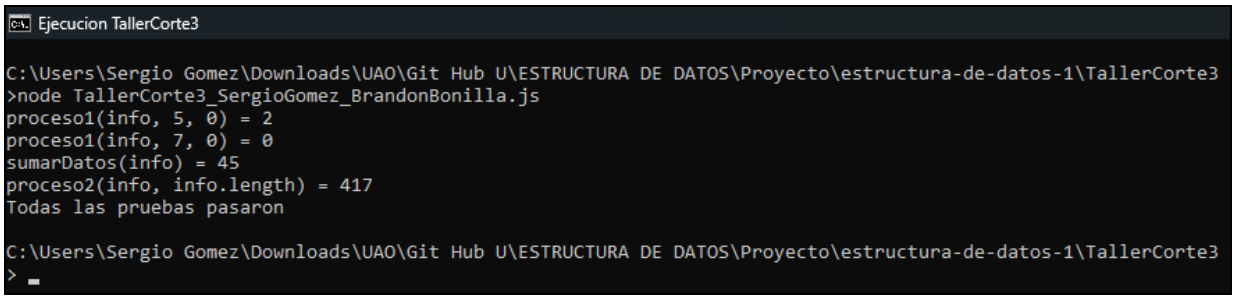


Figura 3. Ejecución de TallerCorte3_SergioGomez_BrandonBonilla.js en la consola.